MUSIC AND
ACOUSTIC ENGINEERING

DIGITAL AUDIO ANALYSIS AND PROCESSING

Homework - Report

Giuliano Di Lorenzo
242712
Filippo Longhi
270146

POLITECNICO
MILANO 1863

1 Introduction

The goal of this project is to implement the speech coding and synthesis system of the *Speak & Spell* ™ educational toy of the late 1970s, based on Linear Predictive Coding (LPC). It is a digital technique that allows to obtain a data-compressed representation of speech signals, from which is possible to recreate vocal sounds.

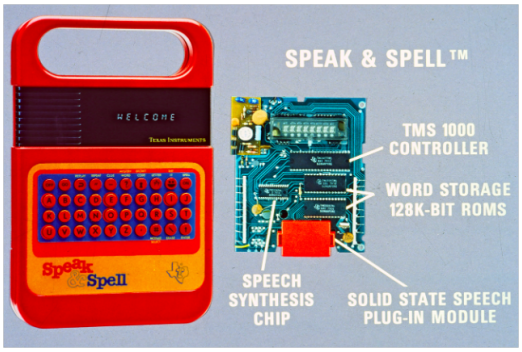

Figure 1.1: Speak & Spell ™

2 LPC theory

The idea behind this encoder is to approximate a signal $s(n)$ using the past p samples:

$$s(n) \approx \hat{s}(n) = \sum_{k=1}^{p} a_k \; s(n-k)$$

In a context where the original signal is the output of a noise-like excitation signal $u(n)$ (with gain G) and a shaping filter $H(z)$:

$$s(n) = \sum_{k=1}^{p} a_k \; s(n-k) + G \, u(n) \quad \Rightarrow \quad S(z) = \sum_{k=1}^{p} a_k \; S(z) z^{-k} + G \, U(z) \quad \Rightarrow \quad H(z) = \frac{S(z)}{G \; U(z)} = \frac{1}{1 - \sum_{k=1}^{p} a_k \; z^{-k}}$$

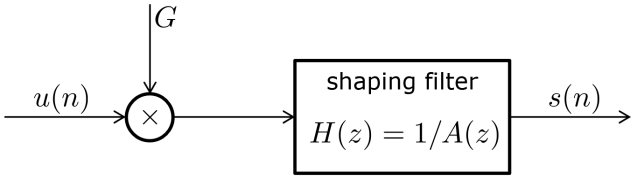

Figure 2.1: Shaping filtering

The prediction error $e(n)$, which is the difference between original and approximated signals, is a scaled version of the excitation signal:

$$e(n) = s(n) - \hat{s}(n) = G \; u(n) \quad \Rightarrow \quad E(z) = A(z) S(z), \quad A(z) = \frac{1}{H(z)} = 1 - P(z)$$

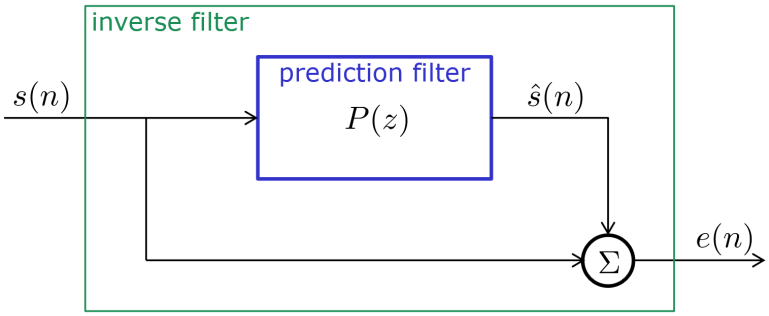

Figure 2.2: Whitening filtering (inverse) and predictive filtering

We obtain the LPC coefficients a_k considering a Wiener filtering problem with $s(n)$ as both input and desired output, in which case the Wiener-Hopf equations become:

$$r(i) = E\left\{s(n)s(n-i)\right\} = \sum_{k=1}^{p} a_k\, r(i-k), \quad i = 1, 2, \ldots, p \quad \Rightarrow \quad \underline{a} = [R]^{-1}\underline{r}$$

$$\underline{r} = \begin{bmatrix} r(1) \\ r(2) \\ \vdots \\ r(p) \end{bmatrix}_{p\times 1}, \quad [R] = \begin{bmatrix} r(0) & r(1) & \ldots & r(p-1) \\ r(1) & r(0) & \ldots & r(p-2) \\ \ldots & \ldots & \ddots & \vdots \\ r(p-1) & r(p-2) & \ldots & r(0) \end{bmatrix}_{p\times p}, \quad \underline{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix}_{p\times 1}$$

Where $r(i)$ is the auto-correlation of the signal $s(n)$. Knowing the LPC coefficients $\underline{a}$ and the original signal $s(n)$, we can compute the prediction error $e(n)$ (using the whitening filter $A(z)$), so that it can be used separately to synthesize the original sound.

3 Data encoding

The first part of the *Speak & Spell* ™ DSP pipeline is to encode some examples of speech files (i.e. single letters or small sentences) and store their LPC models.

3.1 Windowing

The Wiener filtering problem is defined for stationary signals, so we generally want to window s into short frames s_n, so that locally we satisfy this condition. In addition to that, we want to take into account if each frame resembles a voiced or unvoiced sound, so that the LPC model and the synthesis approach change accordingly. We extract the frames using a window $w(m)$ of length w_L and hop-size h:

$$s_n(m) = s\left((n-1)\ h + m\right) \cdot w(m), \quad \begin{cases} n = 1, 2, \cdots, n_{\text{frames}} \\ m = 0, 1, \cdots, w_L - 1 \end{cases}$$

In this project, we implemented the frame extraction using a Hamming window of length $w_L = 256$ samples and hop size $h = 256$ samples, so that overlap does not actually occurs. We tried an additional approach, which was not successful, and that is presented in the conclusions section.

An optional detail, before windowing the signal and extracting the frames, is to apply a pre-emphasis high-pass filter: the goal is to reduce the low-frequency content (that is the range with the most speech energy), so that the resulting spectrum is actually flattened.

3.2 Autocorrelation and LPC coefficients

The encoding starts from computing, for each frame s_n, the autocorrelation $r_n(i)$ and using it to find the coefficients $\underline{a}_n$ that describe the LPC encoder (at frame n) with the Levinson-Durbin recursion algorithm. The `[a,~,~] = levinson(r,p)` function requires $p+1$ values of the auto-correlation (from $r(0)$ to $r(p)$, we take the positive lags only thanks to symmetry) and the LPC order p to compute the coefficients of the whitening filter $A(z)$: we need to exclude the first value (equal to 1) and change sign of the remaining entries to actually obtain the LPC coefficients $\underline{a}_n$.

$$\mathrm{a}_{\mathrm{levinson}} = [1, -a_1, -a_2, \cdots, -a_p] \quad \Rightarrow \quad \underline{a}_n = \{a_k\}_{k=1}^{p} = [a_1, a_2, \cdots, a_p], \quad p = \{4, 10\}$$

3.3 Prediction error and gain

In addition to the encoder coefficients, we want to compute the prediction error e_n of each frame, so that it can be used to describe the excitation for the sound synthesis. We compute e_n by filtering the frame s_n through the whitening filter, whose coefficients are directly obtained by the Levinson function. The gain G_n associated to the frame n is the RMS (root mean square) of the prediction error:

$$A_n(z) = 1 - \sum_{k=1}^{p} a_k \; z^{-k} \quad \Rightarrow \quad e_n(m) = \mathscr{Z}^{-1}\left\{S_n(z) \; A_n(z)\right\} \quad \Rightarrow \quad G_n = \sqrt{\overline{e_n^2(m)}} = \sqrt{\frac{1}{M}\sum_{m=0}^{M-1} e_n^2(m)}$$

3.4 Voiced frame variations

For voiced frames only, we compute the pitch period T_0 using the AMDF technique (Average Magnitude Difference Function). The idea behind this technique is to find the necessary lag i to maximize the auto-correlation $r_{e_n}(i)$ of the prediction error $e_n(m)$:

$$T_0 = \arg \max_{i_{\min} \leq i \leq i_{\max}} r_{e_n}(i) \quad [\text{samples}], \quad \begin{cases} i_{\min} = 25 \\ i_{\max} = 80 \end{cases}$$

We limit ourselves in the range of period $[T_{\min}, T_{\max}] = \left[\frac{i_{\min}}{f_s}, \frac{i_{\max}}{f_s}\right] = \left[\frac{25}{f_s}, \frac{80}{f_s}\right] \approx [3.1, 10]$ ms, considering the usual frequencies range of the voice fundamentals (i.e. $[100, 320]$ Hz).

In order to contextualize (and simplify) this computation, we may to low-pass the prediction error e_n (for example, cutoff at 800 Hz), so that it loses high-frequency useless contributions and makes the error peaks more visible (trying to resemble a train of impulses). This cleaning process helps the pitch period computation. Now, we can use both the prediction error e_n and the computed pitch period T_0 to compute the gain G_n:

$$G_n = \sqrt{T_0 \; \overline{e_n^2(m)}} = \sqrt{T_0 \frac{1}{L}\sum_{m=0}^{L-1} e_n^2(m)}, \quad L = \left\lfloor \frac{w_L}{T_0} \right\rfloor T_0 \leq w_L$$

The idea is to consider the first L samples, where L is the integer multiple of T_0 closest to the frame length w_L. We compute the mean $\overline{e_n^2}$ on these samples and scale it with the pitch period. The last scaling is necessary when dealing with train of impulses: this value of G_n will be the height of a train of impulses, and so the resulting power will be G_n^2/T_0. This observation is discussed in the decoding section.

4 Data decoding

The second part of the *Speak & Spell* ™ DSP pipeline is to use the stored LPC models to synthesize vocal signals, in order to recreate the original audio files.

4.1 Excitation signal

As during the encoding, we approach the reconstruction frame-by-frame at first, then we sum up all frames contributions. We define an excitation signal u and we scale it with the gain G_n, defining a residual signal e'_n that has the same spectrum power of the prediction error e_n:

$$e'_n(m) = G_n \; u(m), \quad \left|E'_n\left(e^{j\omega}\right)\right|^2 = \left|E_n\left(e^{j\omega}\right)^2\right| = G_n$$

In particular, we define u as a train of impulses for voiced frames, with pitch period T_0 (distance between each impulse). Let's remind that the impulses train has total power $1/T_0$, so we need to take into account this scaling factor when computing the gain values for voiced frames (or adjust the impulses height accordingly):

$$e'_n(m) = G_n \sum_i \delta(n - iT_0) \quad \Rightarrow \quad P_{\text{train}} = \overline{e'^2_n(m)} = \frac{1}{T_0} \sum_{m=0}^{T_0-1} e'^2_n(m) = \frac{G_n^2}{T_0} = \frac{T_0 \; \overline{e_n^2(m)}}{T_0} = \overline{e_n^2(m)}$$

On the other hand, we define e'_n as a white noise process for unvoiced frames, with zero mean and variance $\sigma^2 = G_n$:

$$u(m) \sim N(0,1), \quad G_n \; u(m) = e'_n(m) \sim N(\mu = 0, \sigma^2 = G_n) \quad \Rightarrow \quad \left|E'_n\left(e^{j\omega}\right)\right|^2 = \sigma^2 = G_n$$

4.2 Shaping filtering

Once we have defined the residual signal e'_n, we feed it into the shaping filter H_n to reconstruct the frame s_n into the approximation $\hat{s}_n$ such as:

$$\hat{s}_n(m) = \mathscr{Z}^{-1}\left\{E'_n(z) \; H_n(z)\right\}$$

4.3 Reconstructed frames

The last step of the reconstruction is to sum all reconstructed frames $\hat{s}_n(m)$ contributions:

$$\hat{s}(m) = \sum_{n=1}^{n_{\text{frames}}} \hat{s}_n(m) = \sum_{n=1}^{n_{\text{frames}}} \hat{s}\left(m - (n-1) \; h\right) \cdot w\left(m - (n-1) \; h\right)$$

As said in the windowing process during the encoding, the frames do not overlap ($w_L = h = M = 256$ samples, Hamming window), so the summation of the reconstructed frames easily becomes a correct positioning of them:

$$\hat{s}(m) = \sum_{n=1}^{n_{\text{frames}}} \hat{s}\left(m - (n-1) \; M\right)$$

A last optional step, in opposite way with respect to what happens in the encoding, is to apply a de-emphasis low-pass filter, to compensate for the loss of low-frequency energy due to the pre-emphasis filtering.

5 Synthesis results

5.1 LPC models

Let us analyse the characteristics of the LPC model, at first for voiced frames, later for unvoiced frames.

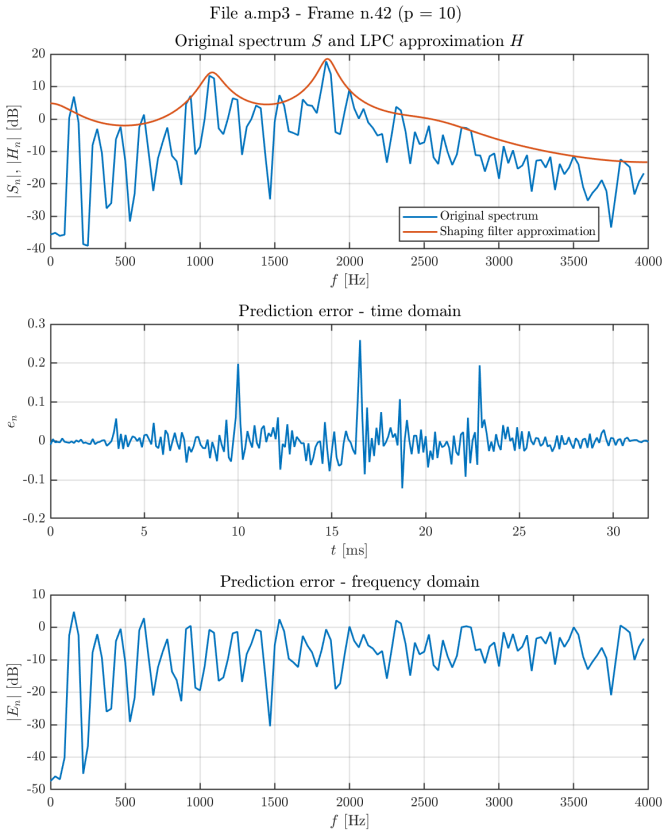

Figure 5.1: LPC modelling for a voiced frame

In the first subplot, we compare the frame spectrum $|S_n|$ with the shaping filter $|H_n|$ that we define when computing the LPC coefficients $\underline{a}_n$ (and the Levinson ones $\mathrm{a}_{\mathrm{levinson}}$). As the name suggests, the shaping filter is the result of a power spectrum envelope estimation: the goal is to define a curve that resembles the original spectrum, taking into account the peaks more than the valleys. In fact, the main peaks of $|H_n|$ are located where some of $|S_n|$ peaks are, and the shapes of the two spectra are quite similar to one another.

Regarding the prediction error, we notice that a few peaks are more peculiar than others, which suggests they may be the ones that define the pitch period of the frame. Knowing that voiced frames are the result of an excitation signal defined as a train of impulses (with pitch period T_0 samples), we want to compute said period from e_n. The problem is that it looks very noisy, and that is the reason why we want to low-pass the prediction error when computing the period: by removing all high-frequency content (in this case after 800 Hz), we evaluate the distance between the main peaks by looking only at the frequency range of interest (as anticipated in the encoding section). By looking at the prediction error spectrum $|E_n|$, we may notice that at low frequencies (i.e. under 500 Hz) a few peaks occur (almost with a degree of regularity): that looks like there is a certain organization in the main frequency tones, which is definitely not typical of a white-like noise process.

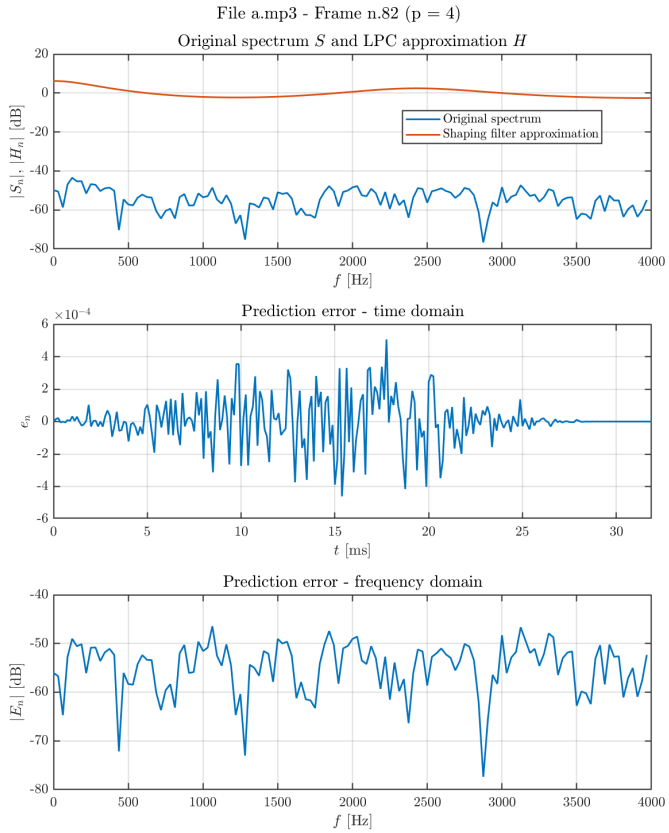

Figure 5.2: LPC modelling for an unvoiced frame

The shaping filter is the result of a power spectrum envelope estimation: the goal is to define a curve that resembles the original spectrum, taking into account the peaks more than the valleys. Like for the voiced frames, the trend of both spectra is quite the same: the obvious difference stands in the vertical shift (different mean dB levels). Although they do not look so much similar as in the previous case, the obtained shaping filter is coherent. The original frame looks like a white-like noise process, having all possible frequencies with similar amplitude values (between -70 and -40 dB, very small amplitudes in every case): that means that no specific frequencies are enhanced (like in the voiced case). Therefore, the shaping filter should be a flat-like curve (which actually is): the difference is that, being a filter that does not change the frequency content of the input signals, the frequency response should be unitary (which means 0 dB).

Regarding the prediction error, we notice a more chaotic nature with respect to the voiced frame scenario, both in time and in frequency domain. The excitation signal for unvoiced frames is a white noise process, so it seems coherent that from e_n we compute the RMS value to define the variance for the excitation signal (which works like a multiplication factor for a zero-mean white noise).

5.2 Excitation signals

Let us now compare the excitation signal generation process, for both voiced and unvoiced frames.

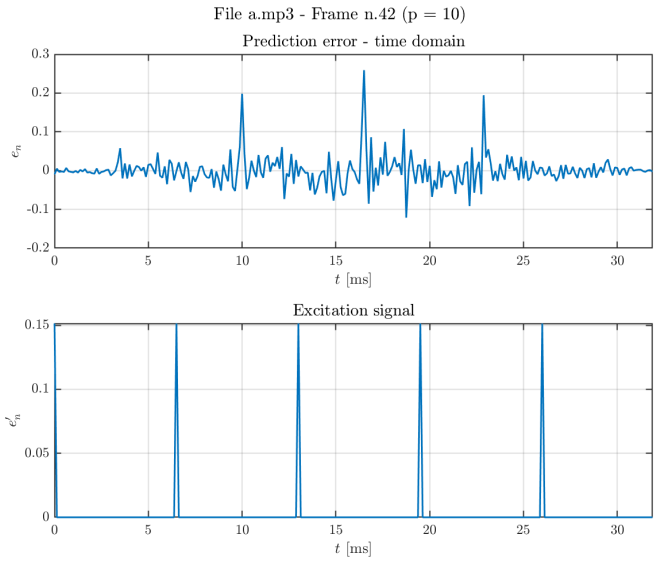

Figure 5.3: Excitation signal for a voiced frame

The excitation e'_n for voiced frames, as anticipated before, is a train of impulses with period T_0 and height G_n. The positioning and the distance between the impulses looks coherent with respect to the reference (prediction error e_n).

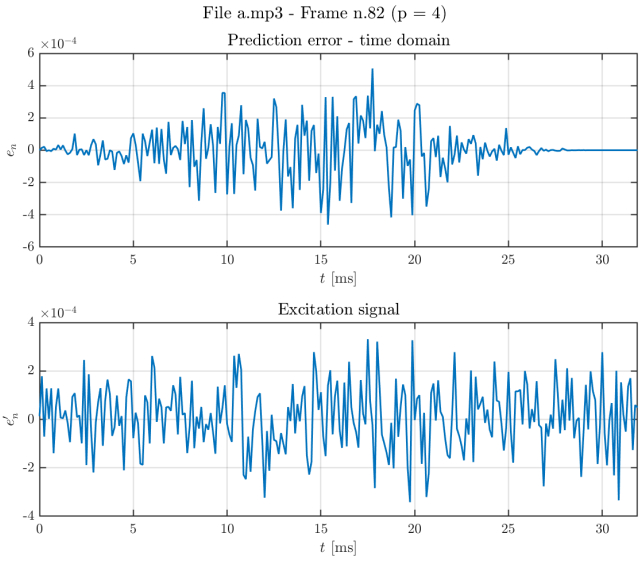

Figure 5.4: Excitation signal for an unvoiced frame

The excitation e'_n for unvoiced frames, as anticipated before, is a white noise with variance G_n. The difference between original signal e_n and artificial one e'_n is that the second one is more compact in a specific amplitude range (being a white noise with multiplication factor G_n), while the first one can be more unbalanced and asymmetric in the amplitude range, although this may not be relevant in some cases (amplitude range has lower order of magnitude with respect to the signal maximum, like 10^{-4} with respect to 10^0).

5.3 Frames and signal reconstruction

Let us finally discuss the synthesis process, by comparing the reconstruction of voiced and unvoiced frames, in addition to the entire signal result.

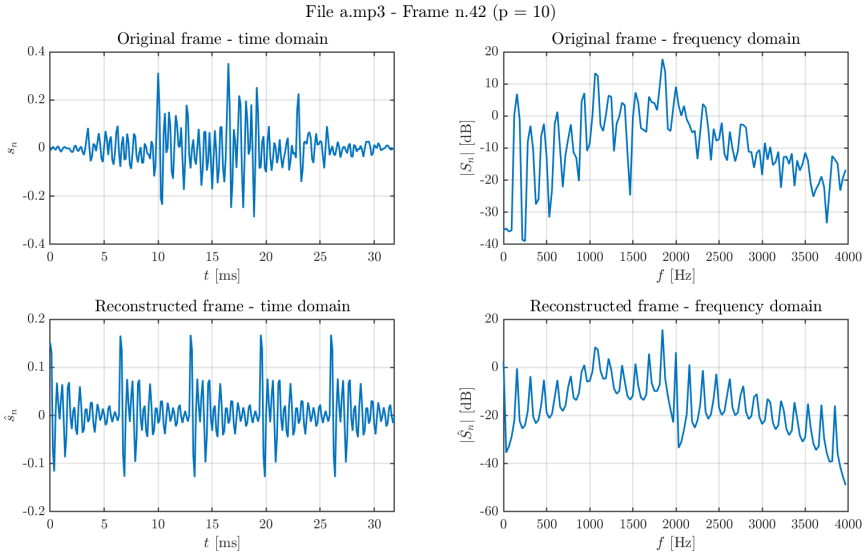

Figure 5.5: Voiced frame reconstruction

The impulsive nature of the voiced frames excitation is still visible in the reconstructed frame $\hat{s}_n$ (as it should be in the original frame s_n). The reconstructed spectrum is way more ideal, having a resemblance of harmonicity, and its envelope reminds of the shaping filter obtained previously.

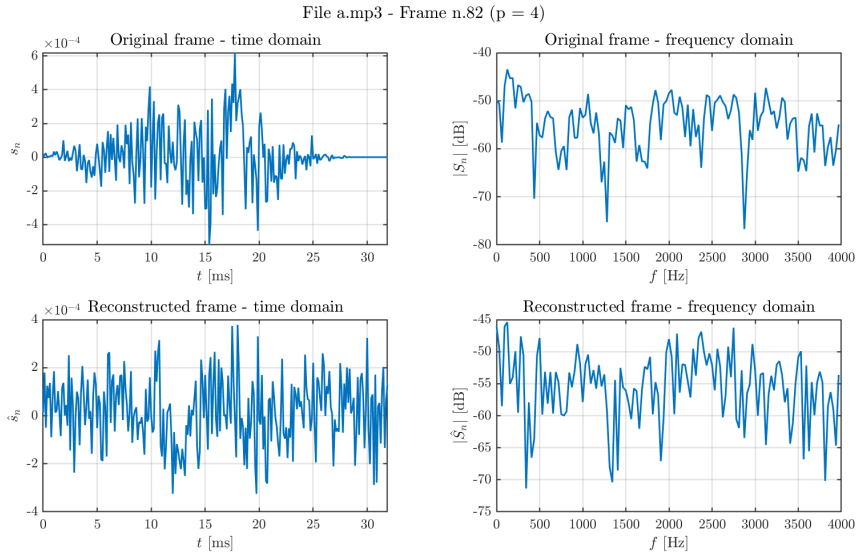

Figure 5.6: Unvoiced frame reconstruction

Both original and reconstructed frames are white-like noise processes, both in time and frequency domain. As in the excitation signal generation, we may notice a difference in the time amplitude range, which is now due to the gain G_n computation in the encoding. In addition to that, the average spectrum dB level seems different from original to reconstructed frame, even though those are always lower than -40 dB (where the level is too low to actually understand any speech, like when whispering).

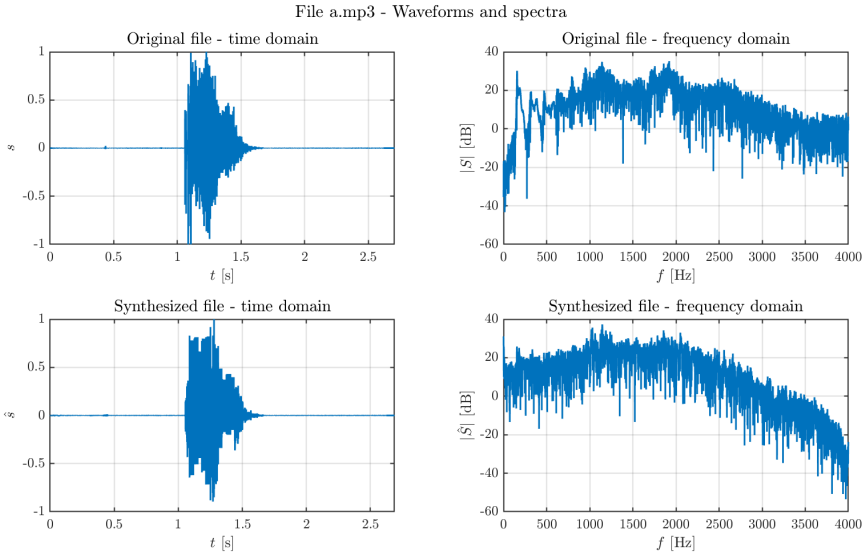

Figure 5.7: Signals comparison

Finally, the overall synthesis process sounds (and looks) quite successful, even though we may do lots of improvements and fine-tunings. The time evolution of the reconstruction is not so much different to the original one. On the other hand, the spectrum changes quite a bit and that is immediately clear when listening to the result: the obtained audio sounds way more noisy and lacks of clarity (in particular for the consonants).

A reconstruction file `"reconstruction.wav"` of `"ces.wav"` is available to be reproduced together with this report.

5.4 Comments about windowing

let us take into account that, in this project, we used a windowing technique with no overlap occurring: this allowed us to take the reconstructed frames and position them in the right time instants.

The secondary approach we tried was using Hanning windowing with 50% overlapping ($w_L = 256$, $h = 128$ samples), which actually satisfies the COLA condition, so that when we add the reconstructed frames they should overlap correctly without any scaling correction. What actually happens, which ruins the final result, is that windowing the reconstructed frames $\hat{s}_n$ (or even the excitation signal e'_n) during the decoding modifies the time content of the frame, and so it does change the frequency content. In particular for voiced frames, the obtained spectrum (that reminded of the shaping filter) is now different. In addition to that, it seems that also in time we may lose information, specifically related to the pitch: the immediate result is that the synthesized sound does not change in pitch during the entire time duration. In some cases, this result may be desired (in artistic or creative contexts), but not for a clean reconstruction of the original speech signal.

A reconstruction file `"bad_reconstruction.wav"` of `"ces.wav"` is available to be reproduced together with this report.